

PODIUM: A Verification-Oriented Open-Source Library for Rendezvous, Proximity Operations, and Docking GNC

Adi Oltean*

July 2026 — v0.1 preprint

Abstract

We present PODIUM, an open-source (MIT) Python library and simulation sandbox for guidance, navigation, and control (GNC) of spacecraft rendezvous, proximity operations, and docking (RPOD) in low and medium Earth orbit, designed from the outset for translation to flight C code validated by abstract interpretation. A survey of the open-source ecosystem finds no maintained tool that combines validated relative-motion dynamics, RPOD guidance primitives, deterministic simulation, and a credible path to verified flight software; PODIUM targets exactly that gap. The library enforces a strict two-layer architecture: a *static subset* algorithm core (pure step functions, fixed array shapes, compile-time loop bounds, machine-readable range contracts on every scalar) mechanically translatable to analyzable C, beneath a full-Python sandbox for truth models and analysis. We describe the implemented v0.1 kernels—Clohessy–Wiltshire and Yamanaka–Ankersen state transitions, a nonlinear dual-propagation truth model with J_2 and differential drag, and pulsed minimum-impulse-bit terminal docking control—together with the validation methodology: independent cross-implementation checks, quadratic-scaling separation of linearization error from implementation error, and CI-enforced validity envelopes for the linear models. A case study validates the docking laws end-to-end against an uncooperative black-box plant, SpaceX’s public ISS docking simulator, via browser automation: system identification recovered an undocumented per-frame actuation model, a sign-inverted rate display, and a frame-rate-dependent control gain, and the campaign surfaced two genuine control defects—a pulse-deadband stall whose standing offset $\tau \Delta v/2$ scales with frame rate, cured by sub-pulse dithering—before the autopilot achieved repeatable docking success. We close with the designed convex guidance stack (lossless convexification on relative-motion STMs, successive convexification for six-degree-of-freedom docking) and the planned contract-to-annotation verification pipeline.

1 Introduction

Autonomous rendezvous, proximity operations, and docking is moving from a capability of a few national programs to a routine commercial requirement: satellite servicing, active debris removal, orbital assembly, and crew and cargo logistics all fly some version of the same terminal problem—bring a chaser from kilometers away to millimeter-per-second contact inside tight lateral, angular, and rate gates [1]. The GNC algorithms involved are mature in the literature, yet the *software* situation is lopsided. Flight-proven RPOD stacks are closed and proprietary. Open-source astrodynamics is rich in propagators and orbit-determination machinery but thin exactly where RPOD

*github.com/adi-oltean/podium. The library and this manuscript were developed with substantial AI assistance (Anthropic Claude); all physics claims are enforced by the repository’s test suite.

lives: validated relative-motion models, approach corridors and keep-out zones, docking-phase control, passive abort safety, and—most conspicuously—any deliberate route from prototype Python to flight-grade, formally analyzable C.

PODIUM [2] is an attempt to fill that gap in the open. Its premise is that verifiability is an architectural property, not a post-processing step: if flight algorithms are written from day one in a restricted *static subset* of Python with contracts as data, the eventual C translation is mechanical, and the resulting code lands in precisely the dialect that industrial abstract interpreters are known to prove runtime-error-free—the approach validated on the ESA Automated Transfer Vehicle docking software by Astrée in 2008 [3].

This preprint documents the v0.1 state of the library. Its contributions:

1. a two-layer, verification-oriented architecture for GNC libraries, with machine-readable interval contracts on every core scalar (Section 3);
2. implemented and cross-validated relative-motion kernels, including a Yamanaka–Ankersen state transition matrix, released under a permissive license together with a truth-model validation methodology and measured validity envelopes (Sections 4–5);
3. pulsed (minimum-impulse-bit) terminal docking control laws with a closed-form deadband-stall analysis, near-zone gain scheduling, and sub-pulse dithering (Section 6);
4. a black-box validation case study in which the docking laws fly SpaceX’s public ISS docking simulator to repeatable success, including the system identification of its undocumented plant and two control defects the campaign surfaced (Section 7);
5. a designed (not yet implemented) convex guidance stack and verification pipeline, stated as such (Section 8).

2 Related work and ecosystem gap

A July 2026 survey of open-source spacecraft simulation and GNC tooling (archived in the repository) examined Basilisk [4], NASA’s 42, Trick, cFS, and GMAT, ESA-adjacent Orekit and tudatpy, the archived poliaastro, nyx-space, and RPOD-specific efforts including AFRL’s CW-based reinforcement-learning environments and the MuJoCo-based SmallSatSim. No maintained tool combines: (i) validated relative-motion dynamics—notably, no surveyed tool ships an open Yamanaka–Ankersen implementation; (ii) docking contact coupled to those dynamics; (iii) RPOD-relevant sensor and actuator models; (iv) GNC algorithm hooks between simulation and flight code; (v) first-class deterministic Monte Carlo; and (vi) a path to verified flight C. Incumbent licenses are fractured (ISC, NOSA, AGPL, “government-purpose”), leaving permissive RPOD territory essentially unoccupied. PODIUM deliberately does *not* compete on general simulation fidelity: Basilisk remains the recommended high-fidelity backend, and Orekit/GMAT serve as cross-validation oracles.

On the verification side, PODIUM follows three precedents: the Astrée/ATV result that abstract interpretation can prove docking flight code free of runtime errors when the code is one synchronous loop of pure, statically-bounded step functions [3]; Feron-style credible autocoding, where controllers carry their invariants as machine-checkable contracts [5]; and the embedded-solver code generation dialect of CVXPYgen and its relatives—static allocation, fixed iteration counts, `math.h` as the only dependency [6]. For closed-loop verification, the ARCH-COMP spacecraft rendezvous benchmark [7] defines the community-standard hybrid model that PODIUM’s mode logic is designed to remain traceable to.

3 Architecture: the two-layer rule

Everything in PODIUM respects one boundary. The **flight core** (`podium.core`, plus flight-side functions in `control` and `guidance`) obeys the *static subset*: pure step functions with no globals, I/O, clock access, or randomness; fixed array shapes known at import time; loops with compile-time constant bounds (solvers are *anytime*: fixed maximum iterations, status code, best iterate); no recursion or function pointers; guarded divisions and square roots; and saturation at every physical boundary. The **sandbox** (truth dynamics, gain synthesis, Monte Carlo, plotting, solver prototyping) may use the full scientific-Python stack. The simulation engine calls flight-core functions through the same signature they will have after C translation.

Contracts are declared once, as data, and consumed four ways:

```
@contract(n=Interval(1e-5, 1e-2), e=Interval(0.0, 0.9))
def stm(n, e, theta0, dt): ...
```

(i) runtime checks during simulation; (ii) comment-annotation blocks (`[range(lo,hi)]`, `[len(6)]`, `PROVE(...)`) for the external abstract-interpretation tool at C translation; (iii) a generated analysis harness that draws inputs nondeterministically from the declared ranges—the analyzer’s operating envelope, as in the ATV and A380 campaigns; (iv) API documentation. Unconstrained scalars are the leading cause of failed interval-domain proofs, so the rule is absolute: every core scalar parameter carries a range contract.

Determinism is likewise non-negotiable: fixed-step integration, one seeded generator, no platform-dependent reductions—identical configuration and seed give bit-identical trajectories, which makes every Monte Carlo outlier exactly replayable.

4 Relative-motion kernels

Frames and conventions are fixed library-wide: target-centered LVLH with x radial (zenith), y along-track, z cross-track; scalar-first quaternions rotating body to reference; SI units throughout.

4.1 Clohessy–Wiltshire

For a circular target orbit with mean motion n , the classical equations [8]

$$\ddot{x} = 3n^2x + 2n\dot{y} + u_x, \quad \ddot{y} = -2n\dot{x} + u_y, \quad \ddot{z} = -n^2z + u_z \quad (1)$$

admit the closed-form state transition matrix implemented in `core.cw`, with two-impulse targeting by inversion of the position–velocity block.

4.2 Yamanaka–Ankersen state transition

For an elliptic target ($0 \leq e < 1$), PODIUM implements the Yamanaka–Ankersen state transition [9] for the Tschauner–Hempel equations. With true anomaly θ as independent variable, $\rho = 1 + e \cos \theta$, and the standard substitution $\tilde{\mathbf{r}} = \rho \mathbf{r}$ with derivatives taken with respect to θ , the in-plane equations reduce to

$$\tilde{x}'' = \frac{3}{\rho} \tilde{x} + 2\tilde{y}', \quad \tilde{y}'' = -2\tilde{x}', \quad \tilde{z}'' = -\tilde{z}. \quad (2)$$

Writing $s = \rho \sin \theta$, $c = \rho \cos \theta$, and $J = k^2(t - t_0)$ with $k^2 = \sqrt{\mu/p^3}$, the implemented in-plane fundamental matrix on $(\tilde{x}, \tilde{y}, \tilde{x}', \tilde{y}')$ is

$$\Phi_f(\theta, J) = \begin{bmatrix} s & c & 2 - 3esJ & 0 \\ c(1+1/\rho) & -s(1+1/\rho) & -3\rho^2 J & 1 \\ s' & c' & -3e(s'J + \sin \theta/\rho) & 0 \\ -2s & e - 2c & -3(1 - 2esJ) & 0 \end{bmatrix}, \quad (3)$$

each column verified by direct substitution into the Tschauner–Hempel system during development. The physical-space transition over a time step is assembled as $\Phi = \Lambda^{-1}(\theta_1) \tilde{\Phi}(\theta_1, \theta_0) \Lambda(\theta_0)$, where Λ maps physical LVLH states to the tilde variables and the anomaly propagation solves Kepler’s equation with a fixed twenty-iteration Newton recursion (the denominator $1 - e \cos E \geq 1 - e$ is bounded away from zero by the eccentricity contract $e \leq 0.9$)—bounded iteration being a static-subset requirement, not a style choice.

4.3 Validation methodology and validity envelopes

Every physics kernel in PODIUM must carry a “receipt”: a test against an independent reference with a tolerance derived from physics rather than tuned to pass. The YA kernel’s receipts illustrate the methodology:

1. **Independent truth.** The STM is compared against numerical integration of the *nonlinear* elliptic relative dynamics (Section 5) across $e \in \{0, 0.05, 0.15, 0.35\}$.
2. **Error-scaling separation.** The residual against truth must scale *quadratically* with separation (halving the initial offset must quarter the error): linearization error is $O(\|\delta \mathbf{r}\|^2/a)$, while any implementation defect scales linearly. This single test discriminates “model error, as designed” from “bug.”
3. **Structural identities.** Exact reduction to the CW STM at $e = 0$; composition $\Phi(t_0 \rightarrow t_2) = \Phi(t_1 \rightarrow t_2)\Phi(t_0 \rightarrow t_1)$; inverse via negative time steps; $\det \Phi = 1$.
4. **CI-enforced envelopes.** The measured one-orbit position error obeys $\varepsilon \leq C(e) \|\delta \mathbf{r}\|^2/a$ with $C \approx 40$ for $e \leq 0.05$ and $C \approx 200$ at $e = 0.2$ (the nonlinear term is amplified near perigee), asserted in CI together with the quadratic scaling so the bound cannot silently loosen. For context, at 1 km separation and low eccentricity this is centimeter-level model error per orbit. The same harness documents *why the YA kernel exists*: at 1 km separation the CW error exceeds the YA error by a factor >30 at $e = 0.05$ and >100 at $e = 0.2$.

5 Nonlinear truth model

The truth layer (`dynamics.nonlinear`) makes no relative-motion approximation: both spacecraft are integrated in ECI under identical force models—two-body, J_2 , and exponential-atmosphere drag with a co-rotating atmosphere and per-spacecraft ballistic coefficients—and the relative state is formed by differencing in the target’s LVLH frame. Double-precision differencing error at LEO radii ($\sim$ nanometers) is far below truth-model needs. The LVLH frame’s angular velocity is treated exactly,

$$\boldsymbol{\omega}_{\text{LVLH}} = \left[\frac{r}{h} (\mathbf{a}_{\text{pert}} \cdot \hat{\mathbf{z}}), 0, \frac{h}{r^2} \right], \quad (4)$$

including the out-of-plane precession component that is often dropped; a dedicated test central-differences the position history under active J_2 and drag and requires agreement with the reported relative velocity to better than 2×10^{-5} m/s. Further receipts: secular RAAN drift matching $-\frac{3}{2}nJ_2(R_E/p)^2 \cos i$ over ten orbits; drag semi-major-axis decay matching the analytic per-orbit estimate; differential-drag along-track drift with the correct sign; two-body energy conservation to 10^{-10} relative at the CI step size. The YA and truth implementations are fully independent, so they cross-check each other.

6 Pulsed terminal docking control

The docking phase of many vehicles—and, conveniently, the control model of the simulator in Section 7—is *pulsed*: each thruster firing imparts a fixed velocity increment Δv (a minimum impulse bit, MIB), and the controller chooses integer pulse counts. `podium.control.docking` provides three laws:

Closing-rate profile. $v_{\text{cmd}}(r) = \text{clamp}(r/\tau_a, v_{\text{contact}}, v_{\text{far}})$: an exponential-decay glideslope in range with a firm, capture-safe contact floor.

Gain-scheduled proportional nulling. Lateral offsets and attitude errors are driven by saturated proportional velocity/rate commands $u = -\text{clamp}(p/\tau, \pm u_{\text{max}})$ with a shorter time constant inside a near zone. The scheduling is not cosmetic. With pulse quantization and a half-pulse deadband, a proportional law stalls once $|p|/\tau < \Delta v/2$, leaving a standing offset

$$p_{\text{stall}} = \frac{1}{2} \tau \Delta v. \quad (5)$$

With far-field gains alone this evaluates to 0.6 m laterally and 0.25° per axis at the case study’s pulse sizes—*outside* the 0.2 m/0.2° docking gates. The near-zone constants place the stall at 0.1 m and 0.05° . PODIUM’s click-dynamics unit tests (a per-axis integrator plant actuated only by fixed increments) caught exactly this failure in the first implementation; the tests encode the sim’s success gates as assertions.

Pulse allocator. $n = \text{round}_0((v_{\text{cmd}} - v_{\text{est}})/\Delta v)$ with a half-pulse deadband (round-toward-zero) so the vehicle never chatters about its setpoint, and a per-cycle saturation. For fine tracking the deadband is optionally replaced by *sub-pulse dithering*: the fractional pulse is carried into the next control cycle (first-order $\Sigma\Delta$ modulation), so the average applied rate tracks commands far below one pulse per cycle. Section 7 shows why this matters.

7 Case study: black-box validation against the SpaceX ISS docking simulator

SpaceX’s public ISS docking simulator [10] implements Crew Dragon’s manual docking mode: pulsed translation and rotation inputs, HUD telemetry, and the operative gates (< 0.2 m lateral, $< 0.2^\circ$ per axis, < 0.2 m/s closing at contact). As an *uncooperative, undocumented plant* with a real success criterion, it makes an unusually honest validation target. A Playwright-driven harness injects the exact autopilot script that human users paste into the browser console; the autopilot reads the sim’s telemetry and calls the sim’s own control handlers (identical to clicking its buttons).

7.1 System identification

Because nothing about the plant is documented, the control mapping was recovered empirically (button-probe campaigns sampling raw telemetry) and by reading the minified source. Findings, all load-bearing: (i) translation state is meters *per frame*, applied along *capsule body* axes, one `translationPulseSize` per click, with the displayed (x, y, z) mapping to body $(-z, +x, +y)$ and displayed lateral values being the *negative* of the port’s body coordinates; (ii) the attitude “rate” display is a click counter whose sign is *inverted* relative to the true error drift—the camera’s angular velocity is proportional to *minus* the accumulated click register, with a frame-rate-dependent gain; (iii) the docking contact test is a raycast whose reach shrinks to 0.1 m at terminal range, so an approach faster than ~ 0.1 m/frame *tunnels through the port geometrically*—observed directly: an otherwise perfect approach at 0.159 m/frame passed through the station untouched; (iv) the success gate requires each world-frame velocity component below 0.004 m/frame with all three angles below 0.2° at contact.

The flight autopilot therefore counts `requestAnimationFrame` to convert its SI control laws to frame units at the actuation boundary, measures the attitude plant gain at engagement with a five-pulse probe (re-estimated in flight), and creeps the final meters at 0.003 m/frame.

7.2 Failure modes found by test

The campaign surfaced three defects, each with a lesson:

1. **Polarity inversion** (all three rotation axes): assumed button directions were backwards, producing positive feedback and an uncontrollable spin in the first live run. In-flight sign-learning proved noise-unstable; the cure was offline system identification and hard-coded, verified polarities.
2. **Lateral limit cycle into structure**: aggressive lateral capture near the station produced ± 6 m oscillations at one-cycle latency and a grazing contact with station structure at ~ 8 m range. Cured by distance-scheduled lateral velocity caps and a 10%-half-angle approach corridor that pauses closing whenever the corridor is violated, with a gate-hold inside 4 m.
3. **Frame-rate-dependent deadband stall**: after headless verification passed end-to-end, a live 60 fps run docked 0.2–0.3 m below the port center and failed. In frame units the stall radius of Eq. (5) becomes $p_{\text{stall}} = \frac{1}{2}\tau \Delta v_{\text{click}} f$: 0.24 m at $f=60$ fps but only 0.02 m at the harness’s SwiftShader 5 fps—the bug was *invisible at every frame rate the automated tests ran at*. Sub-pulse dithering (Section 6) removes the dead zone at any frame rate; regression and live runs then both dock cleanly.

After the fixes, the autopilot achieves the simulator’s official SUCCESS outcome repeatably: headless (harness-verified, with wall-clock compression by state teleports between independently verified flight phases) and on a 60 fps desktop from the standard 200 m start with randomized initial attitude, in roughly three minutes of flight time. Beyond the anecdote, the methodological point is that *cross-fidelity validation—unit-level click dynamics, slow headless end-to-end, and full-rate live runs—caught disjoint bug sets*; no single tier would have sufficed.

8 Designed: convex guidance and the verification pipeline

Two major subsystems are designed and documented in the repository but not yet implemented; we state them here for completeness and to invite scrutiny.

Layered convex guidance. Layer 0 poses far/mid-field planning as one-shot convex programs on the exact CW/YA discretizations: LP/SOCP/QP costs; approach cones as second-order-cone constraints; rotating-hyperplane keep-out zones; Breger–How passive-safety constraints [11]; and thrust-annulus lossless convexification following the G-FOLD lineage [12, 13], whose rendezvous exactness was established by Lu and Liu [14]—with validity checks (controllability, normality, discrete-time repair) shipped as part of the implementation rather than left to the user, since keep-out zones are not losslessly convexifiable [15] and interval-active state constraints weaken the guarantees. Layer 1 adds successive convexification for six-degree-of-freedom docking: penalized-trust-region iteration [16] as the default engine (flight pedigree via NASA SPLICE [17]), SCvx* [18] where a feasibility guarantee is required, and state-triggered constraints for plume and minimum-impulse-bit logic [19]. The prototype-to-embedded path is CVXPY with disciplined parameterized programming, generating static-memory, fixed-iteration C solvers [6]—the same dialect the verification pipeline expects, with onboard feasibility anchored by [20, 21].

Verification pipeline. The static subset plus contracts are designed so that a small AST-walking emitter can translate the core to C with contracts rendered as analyzer annotations and `prove()` calls as proof obligations, validated externally by an abstract-interpretation tool in the Astrée tradition [3]; golden-vector equivalence between the Python and C artifacts validates the translation itself. Closed-loop safety is planned as reachability regression on the ARCH-COMP rendezvous benchmark [7], keeping the code-level mode machine traceable to the verified hybrid model, in the spirit of proof-carrying control [5].

9 Conclusion

PODIUM v0.1 demonstrates that a GNC library can be simultaneously open, physically rigorous, and verification-oriented without exotic tooling: the discipline is architectural. The implemented kernels ship with receipts—-independent truth models, scaling-law bug discriminators, and CI-enforced validity envelopes—and the docking control stack has been validated end-to-end against an adversarially undocumented external plant, an exercise that surfaced exactly the class of unit-convention and quantization bugs that destroy real rendezvous missions. The near-term roadmap is the sim engine and Monte Carlo harness, the Layer-0 convex guidance stack, and the C emitter that turns the contract discipline into proofs. Contributions are welcome; every physics claim in the repository is gated by the test suite, and the authors would particularly value independent replication of the validity envelopes and adversarial review of the static-subset rules.

References

- [1] Wigbert Fehse. *Automated Rendezvous and Docking of Spacecraft*. Cambridge University Press, 2003.
- [2] Adi Oltean. Podium: Physics-precise RPOD GNC library and simulation sandbox, 2026. <https://github.com/adi-oltean/podium>, MIT license.
- [3] Daniel Kästner, Stephan Wilhelm, Stefana Nenova, Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, and Xavier Rival. Astrée: Proving the absence of runtime errors. In *Embedded Real Time Software and Systems (ERTS 2010)*, 2010.

- [4] Patrick W. Kenneally, Scott Piggott, and Hanspeter Schaub. Basilisk: A flexible, scalable and modular astrodynamics simulation framework. *Journal of Aerospace Information Systems*, 17(9):496–507, 2020.
- [5] Timothy Wang, Romain Jobredeaux, Heber Herencia-Zapana, Pierre-Loïc Garoche, Arnaud Dieumegard, Eric Feron, and Marc Pantel. From design to implementation: An automated, credible autocoding chain for control systems. In *Advances in Control System Technology for Aerospace Applications*, 2013. arXiv:1307.2641.
- [6] Maximilian Schaller, Goran Banjac, Steven Diamond, Akshay Agrawal, Bartolomeo Stellato, and Stephen Boyd. Embedded code generation with cvxpy, 2022. IEEE Control Systems Letters 6:2653–2658, doi:10.1109/LCSYS.2022.3173209.
- [7] Nicole Chan and Sayan Mitra. Verifying safety of an autonomous spacecraft rendezvous mission. In *ARCH17: 4th International Workshop on Applied Verification of Continuous and Hybrid Systems*, 2017. arXiv:1703.06930.
- [8] W. H. Clohessy and R. S. Wiltshire. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences*, 27(9):653–658, 1960.
- [9] Koji Yamanaka and Finn Ankersen. New state transition matrix for relative motion on an arbitrary elliptical orbit. *Journal of Guidance, Control, and Dynamics*, 25(1):60–66, 2002.
- [10] Space Exploration Technologies Corp. ISS docking simulator, 2020. <https://iss-sim.spacex.com/>.
- [11] Louis S. Breger and Jonathan P. How. Safe trajectories for autonomous rendezvous of spacecraft. *Journal of Guidance, Control, and Dynamics*, 31(5):1478–1489, 2008.
- [12] Behçet Açıkmeşe and Scott R. Ploen. Convex programming approach to powered descent guidance for mars landing. *Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366, 2007.
- [13] Behçet Açıkmeşe and Lars Blackmore. Lossless convexification of a class of optimal control problems with non-convex control constraints. *Automatica*, 47(2):341–347, 2011.
- [14] Ping Lu and Xinfu Liu. Autonomous trajectory planning for rendezvous and proximity operations by conic optimization. *Journal of Guidance, Control, and Dynamics*, 36(2):375–389, 2013.
- [15] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, Thomas Lew, Riccardo Bonalli, Marco Pavone, and Behçet Açıkmeşe. Convex optimization for trajectory generation: A tutorial on generating dynamically feasible trajectories reliably and efficiently. *IEEE Control Systems Magazine*, 42(5):40–113, 2022.
- [16] Michael Szmuk and Behçet Açıkmeşe. Successive convexification for 6-dof mars rocket powered landing with free-final-time. In *AIAA SciTech Forum*, 2018. arXiv:1802.03827.
- [17] NASA SPLICE project. Dual-quaternion guidance flight results on suborbital vehicles, 2025. AIAA SciTech 2025, NTRS 20240014010.
- [18] Kenshiro Oguri. Successive convexification with feasibility guarantee via augmented lagrangian for non-convex optimal control problems. In *IEEE Conference on Decision and Control*, 2023. arXiv:2304.14564.

- [19] Danylo Malyuta, Taylor Reynolds, Michael Szmuk, Behçet Açıkmeşe, and Mehran Mesbahi. Fast trajectory optimization via successive convexification for spacecraft rendezvous with integer constraints. In *AIAA SciTech Forum*, 2020. arXiv:1906.04857.
- [20] Daniel Dueri, Behçet Açıkmeşe, Daniel P. Scharf, and Matthew W. Harris. Customized real-time interior-point methods for onboard powered-descent guidance. *Journal of Guidance, Control, and Dynamics*, 40(2):197–212, 2017.
- [21] Avishai Weiss, Morgan Baldwin, R. Scott Erwin, and Ilya Kolmanovsky. Model predictive control for spacecraft rendezvous and docking: Strategies for handling constraints and case studies. *IEEE Transactions on Control Systems Technology*, 23(4):1638–1647, 2015.